

Sunny Tea House AI 智能评价生成系统 · 项目复盘笔记

👤 薄皓元 ⌚ 实际耗时：约 50 分钟（借助 AI 编程工具完成） ⚡ 体验地址：sunny-tea.lumiere-private.com

📖 本文档速览：1. AI编程心得 | 2. Prompt调优与现场预案 | 3. Webhook工作流 | 4. 安全与成本 | 5. 交互打磨 | 6. 耗时记录

1. 💡 AI 工具使用心得与提效记录

这次全程用 Cursor / Claude Code 辅助写代码。实际体会下来，大部分时间不是自己一行行敲代码，而是在跟 AI 明确描述需求：让它把前端样板代码、Tailwind 样式和接口骨架快速搭好，我把主要精力放在**业务逻辑的闭环、跨语言 Prompt 调优以及异常防御**上。

- 接口与数据结构定义**：先让 AI 把 TypeScript 类型和 Webhook JSON 格式定清楚，前后端传参一次性对齐，避免了来回改字段名。
- 移动端 CSS 样式调试**：手写移动端自适应和 Apple HIG 规范（点击区域 $\geq 44\text{px}$ ）以前可能要调一两个小时，这次让 AI 直接出 Tailwind 类名，10 分钟左右就把浅灰背景（`#F4F4F5`）和纯白卡片的间距与留白调顺了。
- 接口容错与流式联调**：在联调接口报错时，直接把错误日志丢给 AI，能很快定位到是数据分块还是 JSON 解析的问题。

2. 🎯 跨平台 Prompt 设计、踩坑与调优

2.1 踩坑记录：跨语言标签污染问题

遇到的问题 (V1.0)：一开始直接把前端用户选中的中文标签（如 `['服务好', '出餐快']`）拼进英文 Prompt 里丢给大模型，结果模型生成的英文评价里夹杂着中文，比如输出：“The 服务好, 出餐快 really stood out...”。

原因：大模型在缺少明确的多语言引导时，会把上下文里的非英文词当成专有名词原样保留。

解决办法 (V3.0 定稿)：

- 后端做一层语义映射字典**（`TAG_EN_MAP`）：在传给大模型前，先用字典把中文标签转成地道的英文短语（例如 `'服务好' -> 'friendly and welcoming staff'`，`'出餐快' -> 'super fast turnaround'`）；
- 在 System Prompt 里加硬约束**：明确写死 `"You must output ONLY in natural, fluent American English. NEVER output any Chinese characters."`；
- 后端正则保底**：返回前如果检测到中文字符，自动走保底纯英文逻辑，避免混杂中文。

2.2 跨平台 Prompt 差异化设计

维度	🌐 Google Review (北美本地口碑)	📌 小红书 (种草笔记)
目标受众	北美本地居民、Google Maps 真实食客	湾区华人、留学生探店群体
语言与语气	地道美式英语，客观、冷静，普通消费者口吻	中文简体，热情、闺蜜安利感、带网络化情绪
避坑约束	严禁生硬机器翻译词（如 <code>"I had the pleasure..."</code> ）和中文	严禁生硬大段文字堆叠，必须空行留出呼吸感
标志元素	常用口碑词（ <code>"Super chewy boba"</code> , <code>"Fast turnaround"</code> ）	灵动 Emoji（🥤🌟📸），3个精准标签（#奶茶推荐 #圣何塞美食）
长度控制	控制在 50~70 Words (单词) ，2-3 个短段落	120-150 字，3-4 个精简段落

💡 **长度控制的一个小细节：**为了让 Google 英文评论的字数稳定在 50~70 词之间，我没有用 `max_tokens` 硬截断（因为硬截断容易在句子中间切断导致断句），而是用了 `stop_sequences`（结束序列）配合 Few-shot 样本，让模型在完整的句号和换行处自然停下来，保证句子读起来是完整的。

2.3 线上复盘与现场调 Prompt 的预案

代码里把 Prompt 逻辑全部拆到了 `src/app/api/generate/route.ts` 头部，复盘现场如果需要微调，改起来很快：

- 如果要求“改成带建议的温和差评”：只需在 System Prompt 里把 Tone 改为 *"Constructive & polite customer feedback, mentioning long wait time"*；
- 如果要求“加入特定单品”：代码里入参已经解耦，直接把饮品名传进 `userPrompt` 拼接即可；
- 流式容错：在解析大模型分块时做了边界异常捕获，即使网络偶发抖动也不会导致前端页面卡死。

3. 🤖 附加题：企业微信 Webhook 自动化 workflows

当顾客在前端生成评价后，后端会自动触发一个后台任务：

- 调用大模型提炼出 **20 字以内的中文核心摘要**；
- 生成一段 **50 字左右**给老板的感谢回复草稿；
- 拼装成企业微信群机器人的 Markdown 消息格式，推送到运营群。

💡 **兼顾演示与真实推送的设计：**

- 异步非阻塞：**Webhook 推送用 `try...catch` 包裹在后台异步执行，即便推送失败或网络超时，也不会卡住前端顾客获取评价的主请求（主流程依然秒级返回 200）；
- 环境切换：**如果在环境变量里配置了真实的 `WEWORK_WEBHOOK`，它会真实发到企微信群；如果不配，系统就会在后台记录拼装好的数据，前端底部也做了一个折叠抽屉，点开就能直接看到 AI 提炼的摘要和回复草稿。

```
{
  "msgtype": "markdown",
  "markdown": {
    "content": "### 🍵 Sunny Tea House 新评价通知\n> **来源平台**: 小红书\n> **用户标签**: 服务好 / 出餐快\n\n** 📄 评价内容**"
  }
}
```

4. 🔑 API Key 安全与用量成本

🔒 Key 防泄漏做法

在前端直连大模型容易在浏览器 Network 面板里暴露 API Key。这里通过 **Next.js Serverless 路由做了一层后端代理**：客户端只把参数传给服务端，服务端在环境变量 `process.env.GROQ_API_KEY` 里读取 Key 调用大模型，浏览器端抓包完全看不到 Key。

⚡ 为什么选 Groq

Groq 的首字延迟（TTFT）基本在 **200~300ms**，点击生成后几乎是秒出字，移动端体验很顺畅；每天有 **14,400 次免费调用额度**，对于 Demo 和后续测试完全够用，不需要担心成本超支。

5. 📱 移动端交互与细节打磨

- **标签防误触**：选满 2 个标签后，其他未选中的标签会自动置灰（`opacity-40` 且禁止点击），一眼就能看出不能再选了。
- **生成加载态**：点击生成按钮后立刻变更为 `⌛ AI正在思考中...` 并且处于禁用状态，避免用户连点触发多次请求。
- **中英文分立统计**：Google 模式下按英文单词（Words）统计，小红书模式下按中文字符统计，符合各自的使用习惯。
- **一键复制与跳转**：点击复制后弹出绿色 Toast 提示（`✅ 已复制到剪贴板，即将跳转...`），并在后台自动唤起小红书或打开 Google Maps 真实评价页面。
- **高度自适应**：文本框根据内容多少用 `scrollHeight` 自动撑高，避免在手机上出现难看的内部滚动条。

6. 🕒 实际耗时记录

环节	手工大概要花的时间	这次借助 AI 的实际耗时	做的事情
需求拆解与架构设计	约 40 分钟	约 10 分钟	梳理 5 个关键点，定下 Next.js + Serverless 结构

环节	手工大概要花的时间	这次借助 AI 的实际耗时	做的事情
Prompt 编写与中英调试	约 60 分钟	约 15 分钟	调优去 AI 味，解决中文标签污染，加上 <code>TAG_EN_MAP</code> 字典
移动端 H5 界面与样式	约 90 分钟	约 10 分钟	用 Tailwind 调好移动端适配、自适应高度和按钮热区
接口开发与 Webhook	约 50 分钟	约 10 分钟	写好后端代理、错误兜底和企微 Markdown 消息组装
联调、加固与域名部署	约 30 分钟	约 5 分钟	Vercel 关联 GitHub，绑定独立域名上线验证
合计	约 4.5 小时	约 50 分钟	整体流程顺畅，按时完整交付